Projektgrundlage · AboLife · macOS V1

AboLife

Professionelle Projektdatei für eine persönliche macOS-App als ruhiges Entscheidungs-Dashboard für Ernährung, Training, Finanzen, Gewohnheiten und Wochenreview.

Plattform	Nutzung	Kern	V1-Regel
Native macOS-App	Nur privat für Arber	Dashboard + Quick Add	Lokal, offline, fokussiert

Merksatz
AboLife soll nicht dein Leben vollständig verwalten. AboLife soll dir jeden Tag zeigen, was gerade wichtig ist.

Inhaltsübersicht

01 Projektübersicht

02 Kernidee von AboLife

03 Problem, das die App löst

04 Zielgruppe: nur persönliche Nutzung

05 Was an der Idee schlau ist

06 Was riskant oder unschlau ist

07 Beste Vorgehensweise

08 MVP-Abgrenzung

09 Empfohlener Tech-Stack

10 Architekturvorschlag

11 Datenmodell-Übersicht

12 UI/UX-Richtung

13 Dashboard-Konzept

14 Visualisierungs-Konzept

15 Telegram-Idee und spätere Erweiterungen

16 Roadmap nach Phasen

17 Risiken und Gegenmaßnahmen

18 Codex-Workflow

19 Erster sinnvoller nächster Schritt

1. Projektübersicht

AboLife ist eine persönliche macOS-App, die dabei helfen soll, den Alltag strukturierter zu gestalten und bessere tägliche Entscheidungen zu treffen.

Die App soll nicht einfach nur viele Daten sammeln, sondern dir jeden Tag schnell zeigen, was gerade wichtig ist.

Bereich	Beschreibung
App-Typ	Native private macOS-App
Hauptnutzen	Persönliches Dashboard für bessere Alltagsentscheidungen
Fokus	Ernährung, Training, Finanzen, Habits, Wochenreview
Datenhaltung	Lokal, offline, privat
KI	Optional später, nicht Kern der App
Hauptgefahr	Zu viele Features in Version 1

2. Kernidee von AboLife

> Kernidee
> AboLife ist ein ruhiges persönliches Entscheidungs-Dashboard, das wenige wichtige Lebensbereiche zusammenführt und täglich zeigt, was gerade wichtig ist.

Nicht die beste Definition	Bessere Definition
Ich baue eine App mit Kalorientracker, Finanzplaner und Trainingsplaner.	Ich baue ein persönliches System, das mir täglich hilft, bei Ernährung, Training, Gewohnheiten und Finanzen auf Kurs zu bleiben.

```
Leitfrage:
Was ist heute wichtig, damit ich bei Ernährung, Training,
Gewohnheiten und Finanzen auf Kurs bleibe?
```

3. Problem, das die App löst

Bereich	Typisches Problem
Ernährung	Kalorien und Protein werden nicht dauerhaft sichtbar.
Training	Kein klarer Tages- oder Wochenstatus.
Finanzen	Ausgaben sind nicht direkt mit Monatszielen verbunden.
Gewohnheiten	Erledigt / nicht erledigt wird nicht sichtbar genug.
Wochenrückblick	Man vergisst, Muster zu erkennen.
Motivation	Man baut Tools, aber ändert nicht automatisch Verhalten.

Kernproblem
Zersplitterte Selbststeuerung. AboLife soll diese Bereiche zusammenführen, ohne wie ein überladenes Control Panel zu wirken.

4. Zielgruppe: nur persönliche Nutzung

Die Zielgruppe für Version 1 ist ausschließlich Arber. Die App wird nicht für Öffentlichkeit, App Store oder andere Nutzer entwickelt.

Vorteil	Bedeutung
Kein Account-System	Weniger Komplexität
Kein Billing	Kein Geschäftsmodell nötig
Kein Rollenmodell	Keine Admin-/User-Struktur
Kein öffentliches Onboarding	App kann direkt auf dich zugeschnitten sein
Lokal-first	Privatsphäre und Offline-Nutzung als Standard

Wichtig
AboLife darf persönlich sein. Das ist kein Nachteil, sondern ein Vorteil.

5. Was an der Idee schlau ist

Punkt	Bewertung	Begründung
Persönliche Nutzung	Sehr gut	Weniger Komplexität, mehr Fokus.
Lokale macOS-App	Sehr gut	Schnell, privat, offline.
Dashboard-first	Sehr gut	Täglicher Nutzen steht im Mittelpunkt.
Quick Add	Sehr wichtig	Reduziert Reibung beim Eintragen.
Ernährung + Training + Finanzen	Gut, aber gefährlich	Stark, wenn sauber abgegrenzt.
Telegram später	Sinnvoll später	Guter Capture-Kanal, aber nicht für V1.
Alles in V1 bauen	Nicht gut	Führt fast sicher zu Scope-Explosion.

Stärkster Teil
Der stärkste Teil von AboLife ist das tägliche Entscheidungs-Dashboard, nicht die Anzahl der Module.

6. Was riskant oder unschlau ist

Wichtigste Warnung
AboLife darf in Version 1 kein Life OS werden.

Risiko	Bewertung	Warum gefährlich?	Gegenmaßnahme
Zu viele Module in V1	Hoch	Jeder Bereich ist fast ein eigenes Produkt.	Harter MVP.
Food-Datenbank zu früh	Hoch	APIs und Datenqualität erhöhen Komplexität.	Manuelle Eingabe + Favoriten.
Telegram zu früh	Hoch	Bot, Server, Parser und Sicherheit nötig.	Erst nach stabiler V1.
UI vor Datenmodell	Hoch	Schöne App ohne stabile Datenbasis wird fragil.	Datenverträge zuerst.
KI zu früh	Mittel	Ablenkung vom Kernnutzen.	Später als Review-Hilfe.

7. Beste Vorgehensweise

```
Dokumentation -> UI mit Mock-Daten -> Datenmodell -> Quick Add -> echte Module
```

Phase	Ziel	Ergebnis
1	Projekt sauber begrenzen	Markdown-Spezifikationen
2	UI-Gefühl prüfen	Dashboard mit Mock-Daten
3	Datenmodell festlegen	Stabile lokale Persistenz
4	Quick Add bauen	Schnelle Eingabe
5	Erste echte Module	Training, Habits, Ernährung
6	Review und Export	Langfristiger Nutzen

8. MVP-Abgrenzung

Muss rein	Sollte rein	Später	Nicht V1
macOS-App SwiftUI	Mahlzeiten-Favoriten	Telegram Bot	Web-App
Dashboard	Workout-Templates	iPhone-App	Banking-Sync
Quick Add	Gewicht manuell	HealthKit	Barcode-Scanner
Kalorien & Protein manuell	Tastenkürzel	KI-Zusammenfassungen	OCR
Training erledigt / Dauer	Monatsübersichten	Food Search	AI Coach
Finance Entries		Cloud Sync	Komplexes Multi-Device-Sync
Habits			
Weekly Review			
Backup / Export			

MVP-Kern
Dashboard, Quick Add, Kalorien/Protein, Training, Finanzen, Habits, Weekly Review, lokale Daten und Backup/Export.

9. Empfohlener Tech-Stack

Bereich	Empfehlung	Begründung
App-Plattform	Native macOS-App	Passt zum privaten Desktop-Dashboard.
UI	SwiftUI	Modern, Apple-like, gut für macOS.
State	Observation	Moderner SwiftUI-State.
Datenbank	SQLite + GRDB	Explizites Schema, Migrationen, gute Aggregationen.
Charts	Swift Charts	Nativ, ruhig, ideal für kleine Trends.
Tests	Swift Testing + XCTest	Unit/Integration + UI-Tests.
Export	SQLite Backup + JSON/CSV	Restore + lesbare Daten.
Planung	Markdown + GitHub Issues	Codex-freundlich und leichtgewichtig.

Empfehlung
AboLife V1 = SwiftUI + Observation + SQLite/GRDB + Swift Charts.

10. Architekturvorschlag



Ordnerstruktur

```
AboLife/
App/
DesignSystem/
Persistence/
Database/
Migrations/
Models/
Repositories/
Queries/
SeedData/
Features/
Dashboard/
Nutrition/
Training/
Finance/
Habits/
Review/
BodyMetrics/
Charts/
Services/
PreviewSupport/
Tests/
```

11. Datenmodell-Übersicht

> Prinzip
> Entries sind die Primärwahrheit. Summen, Trends und Scores werden daraus berechnet.

Modell	V1-Zweck	Wichtige Felder
DailyLog	Tagesnotiz und Tagesanker	date, note, dayRating, createdAt
NutritionEntry	Kalorien und Protein	occurredAt, title, caloriesKcal, proteinG
WorkoutSession	Training erfassen	occurredAt, workoutType, status, durationMin
FinanceEntry	Ausgaben/Einnahmen	amount, currency, direction, category, label
HabitEntry	Gewohnheiten tracken	habitKey, date, valueBool
BodyMetricEntry	Gewicht/Tageswerte	occurredAt, weightKg, dayRating
AppSettings	Ziele und Defaults	kcalTarget, proteinTargetG, monthlyBudget
WeeklyReview	Reflexion	winsText, missesText, nextWeekFocusText

12. UI/UX-Richtung

AboLife soll ruhig, Apple-like, hochwertig, persönlich, datenreich und trotzdem nicht überladen wirken.

Regel	Bedeutung
Eine Karte = eine Hauptaussage	Keine überladenen Karten.
Maximal ein Haupt-KPI pro Karte	Fokus erhalten.
Maximal eine Primärvisualisierung	Keine Chart-Flut.
Dashboard zeigt heute und diese Woche	Keine tiefe Historie im Hauptscreen.
Empty States sind Pflicht	App wirkt auch ohne Daten hochwertig.

Sidebar

```
AboLife
├── Dashboard
├── Ernährung
├── Training
├── Finanzen
├── Gewohnheiten
├── Wochenreview
├── Körperwerte
└── Einstellungen
```

13. Dashboard-Konzept

```
┌──────────────────────────────────────────────┐
| Hero-Zone |
| Daily Core Ring + Tagesstatus + Training |
├──────────────────────────────────────────────┤
| Daily Progress |
| Kalorien, Protein, Habits, Gewicht |
├──────────────────────────────────────────────┤
| Weekly Momentum |
| 7-Tage-Trends, Training, Habit-Streaks |
├──────────────────────────────────────────────┤
| Financial Calm |
| Monatsbudget, Ausgaben, Restbetrag |
├──────────────────────────────────────────────┤
| Action Layer |
| Quick Add, Review-Hinweise, Notizen |
└──────────────────────────────────────────────┘
```

Karte	Hauptaussage
Daily Core Ring	Wie gut läuft der Tag insgesamt?
Kalorien	Wie viel ist heute noch offen?
Protein	Bin ich auf Proteinkurs?
Training heute	Erledigt, geplant oder offen?
Habits	Wie viele Gewohnheiten sind erledigt?
Monatsbudget	Wie viel Geld bleibt diesen Monat?

14. Visualisierungs-Konzept

Bereich	Visualisierung	Zweck
Tagesstatus	Core Ring	Gesamtgefühl des Tages.
Kalorien	Progress Bar + Zahl	Zielerreichung.
Protein	Progress Bar + Mini-Trend	Proteinfokus.
Training	Statuskarte + Wochenpunkte	Konsistenz.
Habits	Dot Grid / Checkmarks	Erledigung sichtbar machen.
Finanzen	Monatsbudget-Balken	Ruhiger Überblick.

```
Protein heute
[██████████▒▒▒▒▒] 112g / 150g

Trend 7 Tage:
▁▃▅▃▆
```

15. Telegram-Idee und spätere Erweiterungen

> Bewertung
> Telegram ist sinnvoll, aber nicht für V1.

Warum interessant?	Warum nicht V1?
/meal Döner 850 kcal 45 protein /workout done push 35min /expense Mensa 4.20 /weight 82.4	Bot-Architektur, laufender Mac oder Server, Security, Parser-Komplexität und Sync-Frage.

Bessere spätere Architektur

```
Telegram Bot
|
▼
Kleiner Bridge-Service
|
▼
Append-only Inbox
|
▼
AboLife importiert Einträge
```

16. Roadmap nach Phasen

Phase	Ziel	Ergebnis
0 - Projektdefinition	Repo, Markdown-Dokumente, Persistenzentscheidung, MVP	Saubere Planungsbasis
1 - UI-Prototyp	SwiftUI-Projekt, Sidebar, Dashboard, Mock-Daten	Visuelles Ziel sichtbar
2 - Persistenz	SQLite/GRDB, Migrationen, Modelle, Tests	Stabile Datenbasis
3 - Kernmodule	Quick Add, Nutrition, Training, Habits, Finance	Tägliche Nutzbarkeit
4 - Review & Trends	Weekly Review, 7-Tage-Trends, Monatsübersicht	Muster erkennen
5 - Stabilisierung	Backup, Export, UI-Polish, Tests	Vertrauen und Qualität
6 - Erweiterungen	Telegram, iPhone, HealthKit, KI, Food Search	Erst nach stabiler V1

17. Risiken und Gegenmaßnahmen

Risiko	Gefahr	Gegenmaßnahme
Scope-Explosion	App wird nie fertig.	MVP_SCOPE.md hart einhalten.
Zu frühe Food-API	Komplexität und schlechte Daten.	V1 nur manuell + Favoriten.
Telegram zu früh	Infrastrukturproblem.	Erst nach V1.
Sync zu früh	Datenfehler auf mehreren Geräten.	V1 lokal-only.
Unklare Migrationen	Datenmodell bricht später.	Ab Tag 1 Migrationen.
Schöne UI ohne Nutzen	App wird Designspielerei.	Jede Karte braucht Entscheidung.
KI zu früh	Ablenkung vom Kern.	KI erst später als Zusatz.
Zu große Codex-Tasks	Chaos im Code.	Kleine Tasks mit Akzeptanzkriterien.
Keine Backups	Datenverlust.	Export und Backup in V1.
App ersetzt Verhalten	Du trackst, aber änderst nichts.	Weekly Review verpflichtend machen.

18. Codex-Workflow

> Grundregel
> Codex soll nicht frei die App bauen. Codex soll kleine, klar begrenzte Aufgaben umsetzen.

Codex bekommt immer

```
1. Ziel
2. Kontext
3. Betroffene Dateien
4. Grenzen
5. Akzeptanzkriterien
6. Was nicht gemacht werden darf
```

Gute Tasks	Schlechte Tasks
Baue nur die ProteinProgressCard mit Mock-Daten.	Mach das Dashboard fertig.
Erstelle nur die GRDB-Migration für FinanceEntry.	Baue die Finanzfunktion.
Füge Empty, Partial und Full Preview-State hinzu.	Implementiere alles für AboLife.

Dauerhafte Codex-Regeln

Regel
Keine neuen Packages ohne Freigabe
Keine Telegram-, KI-, Health- oder Sync-Arbeit in MVP-Tasks
Keine Datenbankänderung ohne Migration
Keine UI-Karte ohne Empty State
Keine persistierten Summen ohne Begründung
Keine stillen Refactors außerhalb des Scopes
Jede neue Feature-View braucht Preview-Daten

Regel
Build darf nicht rot werden

19. Erster sinnvoller nächster Schritt

Nicht zuerst	Sondern zuerst
Xcode öffnen und direkt App bauen	Repo erstellen und Projekt-Spezifikationen anlegen

Erste Dateien

```
PRODUCT_SPEC.md
MVP_SCOPE.md
ARCHITECTURE.md
DATA_MODEL.md
UI_SPEC.md
ROADMAP.md
CODEX_RULES.md
```

Finale Empfehlung
AboLife ist eine starke Idee, wenn Version 1 nicht versucht, alles gleichzeitig zu lösen.

Merksatz
AboLife soll nicht dein Leben vollständig verwalten. AboLife soll dir jeden Tag zeigen, was gerade wichtig ist.